Python External Functions

1 Introduction

This guide explains how to implement an external function in Python. It should be read in conjunction with the example Python modules in the same folder as this file. This guide and the Python and C++ examples are also available on our website – see www.orcina.com/resources/external-function-examples/ – and you might find more recent information there.

See also the Python section of the OrcFxAPI reference documentation, which gives the details of the Python objects that OrcaFlex passes to the Python external function, and also describes how to run OrcaFlex from a Python script. For this reference documentation, open OrcFxAPIHelp.exe (found in the OrcFxAPI sub-folder of your OrcaFlex installation folder) and search for the topic *Python External Functions*. This documentation is also available on our website: www.orcina.com/resources/documentation/.

Writing an OrcaFlex external function in Python is significantly simpler than writing it in C++ or some other fully-compiled language. Python takes care of many complexities such as memory management and type casting (see the OrcFxAPI help for more).

The trade off for this convenience is speed: a Python external function will be slower to run than one written in C++. So Python is less suitable if the function does a significant amount of processing and simulation time is an important consideration. However even if this applies, Python might still be useful, as a convenient interface between OrcaFlex and some compiled code that implements the significant processing part of the calculation.

If a single external function is used to calculate more than one data item then it will be called multiple times per calculation step – once for each data item. For information on such cases, see the Handling Multiple Data Items : MovingAverageSpeedAndDirection and Dynamic Positioning System examples, and the External Function Calling Order section, below.

2 Requirements

The external function examples are updated alongside the releases of OrcaFlex, so a modern installation of both OrcaFlex and a Python version that OrcaFlex supports should be sufficient. The first version of OrcaFlex to support Python external functions was OrcaFlex version 9.5, which required Python 2.6 or later.

To test your setup, open a command prompt and type: `python`. This should open the Python command window (providing Python is on your search path) and show the Python version. Then at the Python prompt type: `import OrcFxAPI` (case-sensitive). If you are then presented with the Python prompt again and no error then the OrcaFlex Python API has been successfully installed. Now you can check your OrcaFlex DLL version by typing `OrcFxAPI.DLLVersion()` at the Python prompt – this should be 9.5 or above.

You will also need a text editor for editing your Python code, preferably one that is sensitive to Python's syntax and formatting. We like Notepad++ (http://notepad-plus-plus.org/) but there are many others.

3 Python External Function Overview

Your Python external function needs to be implemented as a Python class that defines a number of specific methods, such as the `Calculate` method which OrcaFlex will call each time the value is needed. The external function methods that OrcaFlex will call if they are defined are described below and in the OrcFxAPI documentation.

The module containing the external function class can also contain other classes and supporting Python code, and can import other modules.

Once you have written the class you can use it in an OrcaFlex model by specifying it as an external function in your model, by opening the Variable Data form (see the model browser), selecting External Functions and creating a variable data source whose **File Name** is your Python module name and whose **Function Name** is your external function class name.

3.1 External Function Life Cycle

Python external functions are managed by OrcaFlex using the following general life cycle, which begins whenever you run a simulation that uses a Python external function:

- **Prepare** – OrcaFlex loads and initialises a Python interpreter and imports your external function script and the OrcFxAPI Python module into your script's namespace.
- **Initialise** – OrcaFlex creates an instance of each Python external function class used by the model and calls its `Initialise()` method once, if that method exists. If your external function exports external results and implements a `RegisterResults()` method then this will be called before the `Initialise()` method in this phase.
- **Calculate** – At each time step of the simulation, OrcaFlex calls the external function `Calculate()` method (possibly more than once per time step) to obtain the data value for that variable data item. If external results are also calculated in by the external function, then the `LogResult()` method will be called at each logging interval following the Calculate call. For line Bend Stiffness tracking calculations the `TrackCalculation()` method will be called before the call to `LogResult()`.
- **DeriveResult** – If the external function defines external results then it must implement a `DeriveResult()` method. This method will be called when one of the external results is requested. The method is called once for each log sample in the requested query period.
- **Finalise** – When the simulation is reset then OrcaFlex calls the external function `Finalise()` method, if it exists, and then closes the Python interpreter. If the simulation is saved then the `StoreState()` method will be called, if present, before `Finalise()` is called.

3.2 Importing the OrcFxAPI module

You must not *explicitly* import the `OrcFxAPI` module in external function code. This import is performed *implicitly* by the host OrcaFlex process. The import needs to be performed by the host OrcaFlex process to ensure that the imported `OrcFxAPI` module correctly references the host OrcaFlex process. The external function code can access the implicitly imported `OrcFxAPI` module in the usual way. For example:

```
def Initialise(self, info):
    info.period = OrcFxAPI.Period(OrcFxAPI.pnInstantaneousValue)
```

3.3 Output Redirection

In the **Prepare** phase above, the Python `stdout` and `stderr` output channels are redirected to OrcaFlex. Any output to them is then displayed in the External Function Output window in OrcaFlex, so error messages and print statement output will appear in that window, as will any Python error messages.

But note that any errors that occur while OrcaFlex is importing either your script or the OrcaFlex Python API will not appear in that window, since these actions occur before the Python output has been redirected.

For details see the OrcFxAPI documentation.

3.4 External function instances

A new instance of the Python external function class is created for each separate use of the external function by your OrcaFlex model. For example, if the external function class `MyAppliedLoad` is used to calculate all 3 components (X, Y & Z) of an applied load on a 6D Buoy, then OrcaFlex will create three separate instances of the class.

And if we then add a second row to the applied load table, again using this external function for the X, Y & Z forces, then OrcaFlex will now create a total of 6 instances of the `MyAppliedLoad` class for this 6D Buoy – one for each of the 3 components of each of the 2 applied loads.

Data may need to be shared between these instances (for example to avoid duplicating calculations for each component). OrcaFlex provides a mechanism for doing this, described in the examples below.

3.5 Tips for Development and Debugging

- Although OrcaFlex will display Python error messages during runtime, those details will not be displayed for errors during importing the external function module. It is therefore more efficient to debug Python external functions outside of OrcaFlex as much as possible, before using them with OrcaFlex.
- Test import your module from the Python console. Change the working directory to the location of your script and import it from Python – this will flush out syntax errors.
- Create a test harness in Python code that creates an instance of your external function and pass your own info object to it with attribute values designed to test specific parts of your code.

3.6 Performance

Python is an interpreted language that runs more slowly than C or C++ so if performance becomes an issue then care should be taken to ensure that the external function's `Calculate()` method is efficient. Here are some ways of doing this:

- Do as much preparation as possible in the `Initialise()` method, or in the class's `__init__()` method. In the `Initialise`() method, create object attributes containing frequently used constant values (e.g. `self.Period` and `self.ObjectExtra` if they are needed for time history requests to OrcaFlex), so that they can be used in the `Calculate()` method.
- For an external function calculating vector quantities (e.g. applied force and/or moment), OrcaFlex will create multiple instances of your external function class. So to avoid repeated

separate calculations, do all of the calculation in one go when the first component is requested, and save the vectors in `info.Workspace` (which is shared between all instances in this model) before returning the requested component. Then when the other components are requested they can be obtained from `info.Workspace` and returned without repeating any calculation. For an example of this see Dynamic Positioning System below.

4 Simple Examples

The Python code for these simple examples is in the module `CurrentFunctionExamples.py` and you can see this module being used by opening `CurrentExamples.dat` in OrcaFlex.

Both these files are in the same folder as this document – open the model in OrcaFlex and the Python module in your preferred editor. On the variable data form in OrcaFlex there are external function variable data sources set up for each of the examples. And on the current page of the environment data form you can select which variable data source is used for the current speed (and direction, for the last example).

Now use the OrcaFlex Workspace menu to open the default workspace file, which shows windows for the time history of wind speed and current speed and direction. When you run the model, the current during the simulation is calculated by the selected external function, and the resulting current speed and direction are displayed.

4.1 A Trivial Example: IncreasingSpeed

The simplest Python external function script is the Python class `IncreasingSpeed`, which implements just one method, `Calculate`, that sets `info.Value` to give a current speed that is zero in the build-up stage (simulation time less than zero) and rises steadily after that.

Look at the `IncreasingSpeed` class in `CurrentFunctionExamples.py` to see this example, and to see it in action, set the current speed in the OrcaFlex model `CurrentExamples.dat` to use the `Increasing speed` variable data source and run the simulation.

The comments in the `IncreasingSpeed` class explain what the code is doing, and in the OrcaFlex simulation you should see the current speed time history showing zero current until simulation time 0.0, and then rising steadily after that.

For details of the `info` object that is passed by OrcaFlex to all the external function methods, see the OrcFxAPI help.

4.2 Returning Values that Depend on Other Things: WindProportionalSpeed

Suppose we want our surface current speed to depend on the wind speed. Look at the `WindProportionalSpeed` class in `CurrentFunctionExamples.py` to see how this can be done. And to see it in action, reset the simulation and set the current speed to use the `Wind-proportional speed` variable data source and re-run.

The OrcaFlex model uses a spectral wind speed, and the `Calculate` method of the `WindProportionalSpeed` class calls OrcaFlex back to get the instantaneous wind speed and then set the current speed accordingly. This example uses an `Initialise` method, so that it can initialise some variables once at the start, which avoids wastefully setting them at each time step in the `Calculate` method.

The constant parameters that are initialised in the `Initialise` method could be more conveniently specified in the OrcaFlex model, using object tags. See the More Sophisticated Examples section below to see how this is done.

4.3 Modelling History Effects: MovingAverageSpeed

To get a smoother current variation, we might want to use a moving average of the wind speed, rather than the instantaneous wind speed, to determine the current. To do this we need to keep a history of the recent wind speed values, and this means that we also need to store and restore this information if the simulation is paused and saved, to ensure the external function can continue correctly when the simulation is resumed.

The class `MovingAverageSpeed` shows how this is done – see the comments in that class. If the simulation is paused and saved, then the `StoreState` method will be called and anything it puts in `info.StateData` will be stored in the simulation file. Then, when the simulation is re-opened and continued the `Initialise` method will be called again, but this time the attribute `info.StateData` will contain what was stored by `StoreState`, which is used to re-initialise to the state the external function was in when the simulation file was stored.

The `json` module is used to serialise our Python state data into a form suitable to be saved in the OrcaFlex simulation file, and to restore it again when the simulation file reloaded.

4.4 Handling Multiple Data Items: MovingAverageSpeedAndDirection

Often the external function wants to handle more than one data item. For example, for an externally calculated applied load you probably want to set 3 or 6 components (3 components of force, and possibly 3 components of moment too). The `MovingAverageSpeedAndDirection` example shows how to do something similar to this – setting both the current speed and current direction using a single external function class.

In the OrcaFlex model, set both the current speed and direction to use the `Moving average speed & direction` variable data source. When the simulation is run OrcaFlex now creates 2 instances of the `MovingAverageSpeedAndDirection` class, one for current speed and one for current direction. And the class methods will be called twice, once for current speed and once for current direction, in that order. The methods can tell which instance is being called using the `info.DataName` attribute, as shown in the example code.

For this external function we want to do combined speed and direction calculations, and do most of them only once, not once per instance. So in this class each of the methods does the bulk of the work when it is called for current speed, which is called first (see External Function Calling Order below). And in that call the calculated current direction value is saved in the `info.Workspace` attribute, which is shared at the model level and can be accessed by both instances of the class. The `info.DataName=='RefCurrentDirection'` call, for current direction, then just needs to use this shared class variable to set the current direction.

A helper function `GetVector()`, which is outside the class, is used in the calculation. And the `StoreState()` method now needs to store more than one piece of state data, so for clarity it stores a dictionary containing the information it requires using the Python built-in `json` module .

4.5 Externally Calculated Vessel Primary Motion

External functions that calculate vessel primary motion must return the vessel motion in `info.StructValue` instead of `info.Value`. This `StructValue` has attributes `Position`, `Velocity`, `Acceleration`, `Orientation`, `AngularVelocity`, `AngularAcceleration`. All of these except `Orientation` are vectors, represented as Python lists (or tuples can be used). `Orientation` is a matrix whose rows are the unit vectors in the vessel axes directions, expressed as components with respect to global axes direction.

For further information see the `ExternallyCalculatedVesselMotion.py` example and the comments in that module. The OrcaFlex model `ExternallyCalculatedVesselMotion.dat` uses this example to impose externally-calculated vessel motion. Open that model, run the simulation and open the default workspace file (`ExternallyCalculatedVesselMotion_default.wrk`) to see the imposed motion.

4.6 External Results: Vessel Clearance from a Specified Object

OrcaFlex File: `VesselClearanceExternalResult.dat`

Python script: `VesselClearanceExternalResult.py`

This example calculates three external results for the global X, Y and Z position difference between a specified position on a vessel and a specified position on another model object.

The external function is set as a globally applied load on a vessel, although the function class does not implement a `Calculate()` method and so has no effect on the vessel motion. The example demonstrates how to register multiple results from an external function and how to log data required for those results and then return the appropriate result values when requested by OrcaFlex.

To calculate the results, the function needs to know in advance the clearance objects and the body positions on the vessel and the clearance objects so that the necessary global position data can be logged during the simulation. The function allows for several clearance objects and body positions to be specified. In the example model, the object tags for Vessel1 specify two position pairs, named *ShapeClearance* and *BuoyClearance*. When requesting the external result, the `ObjectExtra.ExternalResultText` property should contain one of these names (either *Shape* or *Buoy*) to specify which set of positions to base the results on.

The text format used in this example is JSON (Javascript Object Notation). Python has a built in module, `json`, for reading and writing Python types to string and back and it is a convenient way to format the parameters without having to implement a lot of your own code to convert between text and Python values.

The `Initialise()` method in the external function reads the tags and converts the text data to Python types. The body position data is stored in a dictionary (`self.clearanceObjects`) of `ObjectBodyPosition` instances (a helper class).

The `RegisterResults()` method returns details of the results calculated by this external function to OrcaFlex by setting the `info.ExternalResults` property. The result details are returned as an array of dictionaries, one dictionary for each result. The data required to register a result is explained in the OrcFxAPI help.

When the simulation is running, at every log sample interval the `LogResult()` method will be called. This gives our external function a chance to retrieve instantaneous results from OrcaFlex

and perform any calculations required before logging this data with OrcaFlex. In this example we need to obtain the instantaneous global position data for each vessel body position and clearance object body position defined in the tags. Our log data is a dictionary of vessel / clearance object position tuples, keyed by the name given to the combination in the initialization parameters. This log data is returned as a string to OrcaFlex by setting the `info.LogData` property, again we use `json` as a convenient way to convert this data to a text.

In this example we have only one instance of the external function, set for the Global Applied Force X for the vessel. If we had multiple instances, for example if our function actually modified the applied force in the X, Y and Z directions, then we want to avoid unnecessarily extracting and logging the same data more than once. To avoid this we test the `info.DataName` property before doing any logging.

Once the model is running dynamics, or the simulation has completed then the results we registered in `RegisterResults()` will be available through the OrcaFlex results form. When selecting one of the custom results we need to specify which clearance object we want, we do this by providing one of the names we defined in the tags (either *Shape* or *Buoy* for this example) in the *External result text* field on the Select Results form in OrcaFlex. This will be passed to our external function in the `info.ObjectExtra.ExternalResultText` property in the call to our `DeriveResult()` method when the external result is requested. The `info.ResultID` property will contain the requested result's ID we registered in `RegisterResults()`, and the `info.LogData` property contains our logged data from the `LogResult()` method.

Our `DeriveResult()` method first converts the logged data string back to Python types using json.loads() then looks up the required set of position data based on the value passed in `info.ObjectExtra.ExternalResultText`. Next we check the `info.ResultID` property to see which result value we need to return. The result value is returned by setting the `info.Value` property.

4.7 External Results: Using Track Calculation

OrcaFlex File: `LineStressExternalResult.dat`

Python script: `LineStressExternalResult.py`

This example uses the track calculation feature of the Bend Stiffness variable data source to provide a custom line stress component result. The track calculation feature allows an external function access to a line node's instantaneous calculation data without incurring the performance expense of having to calculate Bend Stiffness within the external function. The external function is specified as a tracking function in the Bend Stiffness data source on the Variable Data Form.

The custom stress component result calculated by this example allows the user to specify tension and curvature stress factors as well as the component angle. To do this the function needs to log the curvature values and wall tension for each half segment. The curvature values are available from the `info.InstantaneousCalculationData` object provided by OrcaFlex to the `TrackCalculation()` method. In this example the line has 50 segments of one line type which results in 100 instances of the external function being created (one instance for each half segment).

Our single result is registered when the method `RegistersResults()` is called, supplying the result name, *External Stress*, and units to appear in OrcaFlex and an ID which OrcaFlex uses to

refer to this result. Although this method will be called once for each instance, OrcaFlex only permits unique result IDs and ignores subsequent registrations. Following the call to `RegisterResults()`, each instance's `Initialise()` method is called; where a working data object (an instance of `NodeHalfSegmentData`) is created. We can't set the arclength in the `Initialise()` method as this data is not provided until the call to the `TrackCalculation()` method.

The `TrackCalculation()` method is called once per log interval, before the call to `LogResult()`. This method is passed the instantaneous data for the half segment in the `info.InstantaneousCalculationData` property. From this we can set the arclength and curvature values in our working data. In the following call to `LogResult()` we query the model to obtain the Wall Tension at the half segment arclength. This data is logged with OrcaFlex by setting the `info.LogData` property and using the Python `json` module to convert the array of tension and curvature to a string.

When requesting this result from OrcaFlex, the user can specify stress concentration factors for tension and curvature, and an angular position. These parameters are passed to the `DeriveResult()` method in the `info.ObjectExtra.ExternalResultText` property. The parameter text needs to be in JSON format for this example. A tension and stress factor of 1.1 and 1.2 respectively with a theta of 45.0 degrees would be represented in the external result text as:

```
{"w": 1.1, "c": 1.2, "t": 45.0}
```

The `DeriveResult()` method retrieves the saved log data for the current sample period from the `info.LogData` property and converts this back into Python values using the `json` module. Next the external result text is also parsed using `json`. Finally the stress result is calculated and returned to OrcaFlex by setting the `info.Value` property.

4.8 External Results: Fatigue

An externally calculated result variable can be consumed by the OrcaFlex fatigue module. This allows you to define a bespoke stress result variable in your Python code and let OrcaFlex calculate and collate fatigue damage.

An example of this can be found in the fatigue file `ExternallyCalculatedStress.ftg`. In order to carry out the fatigue analysis you need to create a load case simulation file. This is based on `LineStressExternalResult.dat` and so you will need to generate a dynamic OrcaFlex simulation file based on that data file.

Once you have generated the simulation file, open the .ftg file in the OrcaFlex fatigue module. Note that the *Damage Calculation* data is set to *Externally calculated stress*. This is the setting that determines that an external results variable will be used.

Now switch to the Components page. This is very similar to the input data for *Stress factor* fatigue. Indeed, the form of our external result was chosen so that we can compare against the built-in stress factor fatigue option.

The *Stress Result* data item is *External Stress*, i.e. the name of our external result variable. The *Component Name* data item is used to specify the external result text. In our example file this is:

```
{"w": 10, "c": 250e3, "t": %theta%}
```

The first and second values are the wall tension and curvature stress factors respectively. The final value is theta. Note that we use the special symbol `%theta%` rather than specifying a fixed value. The reason being that we extract multiple stress results, at different circumferential points, at each arc length along the line. When OrcaFlex calls the external function it replaces `%theta%` with the true theta value, in degrees, and then calls `DeriveResult()`.

You can now calculate the fatigue for this input data and view results just as you would for any other OrcaFlex damage calculation.

Now switch the *Damage Calculation* to *Stress factors*. Note that the data has been setup to match that which we used for externally calculated stress. The tension stress factor is 10 and the curvature stress factor is 250e3. You can calculate fatigue now and confirm that the results are the same as we obtained using externally calculated stress.

One final point to make is that fatigue analysis is one of the most demanding calculations that OrcaFlex performs. Using externally calculated stress can lead to rather slow performance of the fatigue calculation. This is one scenario where the performance benefits of a native external function (see the 'C++ External Function Examples') could out-weigh the convenience of Python.

5 More Sophisticated Examples

The following more complex examples illustrate further aspects of the Python external function interface.

A common element to these examples is reading and using parameters that are specified in the OrcaFlex model in the object tags. These tags are function-specific data for the external function to use in its calculations, and are accessed using the Python mapping object `info.ModelObject.tags`.

The tag values are always given as strings, but we often want numeric values, and possibly want to use default values when the parameter is not specified. The remaining external function examples all use the following convenience functions that provide this functionality:

```
def GetParam(paramName, default=None):
    param = info.ModelObject.tags.get(paramName, None)
    if param is None:
        if default is None:
          raise Exception('Parameter {} is required but is not included in ' \
            'the object tags.'.format(paramName))
        return default
    return param

def GetFloatParam(paramName, default=None):
    return float(GetParam(paramName, default))

myParam = GetFloatParam('ParameterName', 1.234)
```

These functions are passed the parameter name and a default value. If the parameter is defined in the object tags, its value is returned. If the parameter is not defined, and the default value is defined, then the default value will be returned. Otherwise an error is raised.

5.1 Wing Angle Depth Control

OrcaFlex file: `WingAngleDepthControl.dat`

Python script: `WingAngleDepthControl.py`

This example illustrates how external function parameters specified in the OrcaFlex model can be read by the external function and used in the calculations.

The example implements control of the Gamma angle of two wings attached to a towed fish, in order to maintain level flight at a specified depth. Active control only starts once a specified simulation time (`ControlStartsAtTime`) has been reached, and this time, the target depth and other parameters, are all specified in the object tags on the `Towed Fish` data form.

The external function `Initialise()` method reads these parameters, and its `Calculate()` method checks if the simulation time has reached the `ControlStartsAtTime`, and if so calculates the new wing Gamma angle. Before that time the `Calculate()` method leaves the function value unchanged, so the wing Gamma angles remain at the **Initial Value** that is specified on the Variable Data form in OrcaFlex, which is 90 degrees in this example. When the time `ControlStartsAtTime` is reached the control system becomes active and the wing Gamma angles are controlled to make the towed fish dive to and then remain at the target depth.

The control system is a very basic proportional control, i.e. the wing angle is set to a constant multiplied by the error signal (the error signal being the difference between the actual and the target depth). Because the wings are independently controlled, each wing seeks out the target depth, and this gives roll control of the towed fish as well as depth control. A real control system would be more sophisticated than this basic example.

For further details see the comments in the `WingAngleDepthControl.py` module.

5.2 Heave Compensated Winch

OrcaFlex File: `HeaveCompensatedWinch.dat`

Python script: `PIDController.py`

This example models a subsea template being lowered by a heave compensated winch from the stern of a handling vessel. The Python external function that implements the compensation system uses a generic PID controller (Proportional, Integral, Differential) that we have implemented in a fairly general manner, so that it could be used for other control purposes as well, with only small changes.

In OrcaFlex, open the model `HeaveCompensatedWinch.dat`. On the Control page of the Winch data form, the winch Dynamics Mode is set to pay out at a rate determined by `PIDWinchControl`, which is an external function variable data source that is specified, on the Variable Data form, to be the class `PIDController` in the Python module `PIDController.py`.

The winch tags (defined on the data form) contain text strings that specify various model-specific parameters for the controller. These are read by the external function and are used as parameters to the PID controller that controls the winch wire payout.

Run the simulation, and on the Workspace menu select **Use file default** workspace. This sets up windows showing time histories of the winch wire length paid out and resulting Template Z coordinate.

For the first part of the simulation the controller is not active, since the ControlStartTime is specified to be 20 seconds in the winch object tags. So, until simulation time t=20 seconds the winch has a constant length of wire paid out, and so the template rises and falls with the wave-induced vessel motion. At time t=20 seconds, the PID controller becomes active and after that the winch wire is paid out and controlled in order to lower the Template to a Z coordinate of -85m and then keep it steady at that depth.

Controller Theory

The PID controller has the following form:

$$\text{Winch Payout Rate} = k_0 + k_P.e + k_I \int e.dt + k_D.\left(\frac{de}{dt}\right)$$

where

- e = error signal, which in this example is e = TargetValue – Template Z
- k_0, k_P, k_I, and k_D are control parameters of the PID controller, specified in the winch's object tags.

Python Code

For details of how the Python external function works, see the comments in the Python code.

`Initialise()` **method**

This sets up some object variables, `periodNow` and `ObjectExtra`, that will be needed in the `Calculate()` method to ask OrcaFlex for the current value of the controlled variable.

It then gets the parameters, which are specified in winch object tags. If this is a new simulation then `info.StateData` will be `None`, so it initialises the controller. But if a previous simulation file is being opened then `info.StateData` will contain the state that the `StoreState()` method saved, so that state is restored.

`ObjectExtra` contains any extra information that is needed when a result value is requested. In this example the controlled variable is a buoy Z-coordinate, and this depends on the Position on the buoy, which is specified on the OrcaFlex results form when that result is requested. In Python this is defined by `ObjectExtra.RigidBodyPos` (this attribute of ObjectExtra is also used for vessels, which are also rigid bodies in OrcaFlex). Some OrcaFlex result values (e.g. winch tension) do not require any extra information, in which case ObjectExtra is not needed. But others require different extra information to define them – e.g. winch Connection Force requires `ObjectExtra.WinchConnectionPoint` and many line results require `ObjectExtra` attributes such as `NodeNum` or `ArcLength` to be set, in order to define the point on the line at which the result value is wanted.

`Calculate()` **method**

The controller is not active until simulation time reaches the `ControlStartTime` specified in the winch object tags. Before this time the `Calculate()` method does nothing, so the controlled value (winch payout rate) stays at the `Initial Value` that is specified on the Variable Data form in the OrcaFlex model, which is zero.

The OrcaFlex model can perform multiple calculations in a single time step – e.g. if implicit integration is used, since then each time step is an iterative calculation. Because of this the

`Calculate()` method must be careful to only step forward when this is a new time step, which is indicated by `info.NewTimeStep`.

The Python code implements the controller theory given above. The controller needs to maintain calculation data from the previous time step, in order to calculate the integral and differential parts of the control. This is done using two instances (`self.prev` and `self.now`) of a `PIDstate()` helper class. This is a simple class with no methods, and the two instances are created in the `Initialise()` method and used to record data for the current and previous time step.

The $\int e.dt$ and de/dt terms in the control equation cannot be calculated on the first time step of control, since no `self.prev` data is available. So for this first time step these terms are set to the `Initial e/D` and `Initial De` values specified in the winch object tags. These parameters are both set to zero, and that value is probably appropriate in almost all cases.

The time histories of Winch length and Template Z show that the controller does indeed lower the Template to the target depth and hold it there, by paying out or hauling in winch wire to compensate for the vessel heave motion.

`StoreState()` **method**

This method is called if the simulation is saved to a file. It uses the Python `json` module to save the current state of the controller as a string in `info.StateData`, which OrcaFlex saves in the simulation file. When the simulation is re-loaded OrcaFlex calls the external function's `Initialise()` method with this data put back into `info.StateData`, so that the `Initialise()` method can restore the controller state to what it was when the simulation was saved.

5.3 Dynamic Positioning System

OrcaFlex File: `DynamicPositioningSystem.dat`

Python script: `DynamicPositioningSystem.py`

This is an example of a very basic vessel dynamic positioning (DP) system, implemented by applying an externally-calculated Applied Load that models the DP thrusters.

The external function tries to keep the vessel on station by making the global applied force and moment simply proportional to the difference between the actual and target position and heading of the vessel. This has the same effect as applying simple linear springs in the global X and Y-directions, and a torsional spring about the vertical. A real DP system would of course be much more sophisticated than this simple example, and would use multiple thrusters a more complex controlling algorithm.

Open the model `DynamicPositioningSystem.dat` in OrcaFlex. In this model there are 3 vessels, which are initially all on top of each other. The **DP Vessel** (yellow) is the vessel to which the externally calculated DP loads are applied. The **Target Position** vessel (red) is simply a fixed vessel whose purpose is to mark the target position and heading that has been specified for the `DP Vessel`. The **No DP Vessel** (green) is identical to the **DP Vessel** but with no DP system acting, so it shows where the vessel drifts due to wave, current and wind effects if DP is not used.

When you run the simulation the **Target Position** vessel (red) stays in the target position, since it is fixed. But you can see the **No DP Vessel** (green) drift away a lot, and it yaws significantly too.

The **DP Vessel** (yellow) drifts a bit from the target position, but the control system keeps it quite close to the target position and heading.

The model data settings that achieve this are as follows. On the **Applied Loads** page of the **DP Vessel** data form, a global applied load is specified. And this load uses the external function variable data item called `Thruster` to calculate the components of applied force in the global X and Y directions, and the applied moment about the global Z direction (vertically upwards). On the Variable Data form, the Python module and class is specified for this `Thruster` variable data source.

Notice that the same external function (`Thruster`) is specified for both the X and Y applied force components, and also for the Z component of applied moment. The external function calculates all 3 components when the X-component is requested, since that component is called first (see External Function Calling Order below). It then returns each load component separately, one component for each call to its `Calculate()` method.

The target position and heading of the vessel, and two control parameters (kf and km), are specified in the object tags of the DP Vessel. This allows the same external function class to be used for different applications, and allows these model-specific parameters to be specified in the OrcaFlex model.

Python Code

For details of how the Python external function works, see the comments in the Python code.

6 External Function Calling Order

The Handling Multiple Data Items : MovingAverageSpeedAndDirection and Dynamic Positioning System above are examples where a single external function is used for more than one data item in an OrcaFlex model. We refer to this as the externally-calculated data items being 'coupled'.

In the Dynamic Positioning System examples the external function is specified for 3 components (the x force, y force and z moment of the applied load). In this situation the external function will be called multiple times at each calculation point - once for each of the 3 components in the case of externally-calculated applied load. But we want the external function to calculate the whole applied load vector in one go, i.e. all 3 components together and only once, and then simply return the appropriate component when the 3 calls to the function are made.

To make this practical, OrcaFlex sticks to the following rules when calling external functions:-

- For any given externally-calculated data item (e.g. a single component of an applied load), the `Initialise()` call will occur first, then any `Calculate()` calls and finally and `StoreState()` call with any `Finalise()` call last.
- For any given model object, the externally-calculated data items will always be called in the same order each time. But the ordering between different model objects (i.e. different objects in the model browser) is not defined and could change.
- For the following situations, which we expect are the most likely coupling of data items, the order of calls is defined as follows:
- Reference current speed and direction (if both are externally-calculated) will be called in that order, i.e. speed before direction.

- For applied loads on a 6D buoy or a vessel, the external functions will be called in the 'reading' order that they appear on the data form, i.e. left to right, starting at the top of the data form and working down. So global applied loads are called first, starting at the top row and working down, and then any applied loads in the corresponding order. So if all components of both global and load applied loads were externally-calculated, then the order of calls would be:
  Global applied load top row, x-force component, then y-force, then z-force, then x-moment component, then y-moment, then z-moment
  Global applied load 2nd row, x-force component, then y-force, then z-force, then x-moment component, then y-moment, then z-moment
  ...
  Global applied load last row, x-force component, then y-force, then z-force, then x-moment component, then y-moment, then z-moment
  Local applied load top row, x-force component, then y-force, then z-force, then x-moment component, then y-moment, then z-moment
  Local applied load 2nd row, x-force component, then y-force, then z-force, then x-moment component, then y-moment, then z-moment
  ...
  Local applied load last row, x-force component, then y-force, then z-force, then x-moment component, then y-moment, then z-moment
- For applied loads on a node, the external functions are called in the following order:
  x-force, then y-force, then z-force, then x-moment, then y-moment, then z-moment
  The node calling order is not defined (and can change from run to run or even time step to time step). So you cannot assume that the calls for any given node occur before those for higher numbered nodes.
- For externally-calculated line bend stiffness, the order of calls for a given node is:
  'In' bend moment, x-component, then y-component
  'Out' bend moment, x-component, then y-component
  The node calling order is not defined (and can change from run to run or even time step to time step). So you cannot assume that the calls for any given node occur before those for higher numbered nodes.
- For a turbine, the external functions are called in the following order:
  Generator motion controller
  Pitch controller
  Generator torque controller
  Across multiple turbines, the calling order is not defined.

If you want to write an external function that will be used for coupled data items, you should not assume anything else about the order of calls, other than what is defined above. If this is a problem, and you need to know more about the order of calls, then please contact us.

7 Orientation matrices

The instantaneous calculation data objects that are passed to external functions contain orientation matrices. These matrices define the orientation of the body with respect to global

axes directions. Specifically the rows of the orientation matrix are the unit vectors in the body's local axes directions, expressed as components with respect to global axes directions.

For the sake of an example, consider the 6D Buoy's instantaneous calculation data. The orientation matrix is named `OrientationWrtGlobal` and we are also supplied the velocity of the buoy in a vector named `VelocityWrtGlobal`. The velocity is, as indicated in the name, expressed with respect to global axes directions.

Now suppose we wish to express the velocity with respect to the buoy's axes directions. This is achieved by multiplying the orientation matrix by the vector:

```
VelocityWrtBuoy = OrientationWrtGlobal × VelocityWrtGlobal
```

In this equation we treat both vectors as column vectors.

To perform such a re-orientation in Python it is simplest to use the NumPy module. The code to do so would look like this:

```
# import the module, do this at the top of the external function source file
import numpy as np

# later in the external function source file
VelocityWrtBuoy = np.dot(OrientationWrtGlobal, VelocityWrtGlobal)
```

Note that the `dot` method is the standard way to perform matrix/vector multiplication in NumPy.

Now suppose that you wished to perform the re-orientation in the opposite direction. That is to re-orient a vector with respect to body axes directions to a vector with respect to global axes directions. So, for example, consider the vector named `AngularVelocityWrtBuoy`. In order to express this vector with respect to global axes directions we multiply with the orientation on the right hand side:

```
AngularVelocityWrtGlobal = AngularVelocityWrtBuoy × OrientationWrtGlobal
```

In this equation we treat both vectors as row vectors.

The equivalent code in Python is this:

```
AngularVelocityWrtGlobal = np.dot(AngularVelocityWrtBuoy, OrientationWrtGlobal)
```